Theory Of Computation

Basics:

- Decidable prob^m: Algo. exist
- Undecidable " : Algo. ! exist.
- Alphabet (Σ): finite, Non-Empty set of symbol (atomic)
- String: finite Seq^n of Symbols over the given Σ.
  $a^0 = \epsilon$ (zero length)
- formal language: Any set of string over given Σ.
  - finite, empty $= \{\}$
  - infinite, $\{\epsilon\}$ $\{\{\ldots\}\}$
  - complete. $(\Sigma)^*$ → $\{\epsilon, a, b, aa, ab, ba, \ldots\}$

Language	Machine
Regular	D.F.A.
context free	P.D.A
context sensit^v	L.B.A.
Recursive Enume^n	T.M.

Finite Automata:

→ w/o output: DFA, NFA, ϵ-NFA
→ w output: Mealy & Moore Mach^n

DFA: $(Q, \Sigma, q_0, F, \delta)$

→ finite automata in which from every state on every input symb^l exactly one transt should exists.

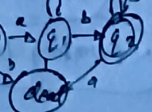
- F: set of final states → any No. (0, 1, 2 ...)
- q_0: only 1 initial " .

Note:
→ If comparison exist b/w Symbols of langu then DFA is not possible.

$L = \{a^n b^n \mid n \geq 1\}$ ✗ $L = \{a^m b^n, m \neq n\}$ ✗

$L = \{a^n b^m \mid m, n \geq 1\}$ ✓ $L = \{a^n b^{n+m} c^m \mid n, m \geq 1\}$ ✗ (dependency.)

* always Start with min' String while Constructing DFA.

$a^n b^m, n, m \geq 1$ ⇒ 

Y DFA ⇒ NFA (conve)

- DFA not exist for Non-Regular language. $L = a^{n^2}, a^{2^n}, a^{\text{Prime}} \ldots\ldots$
  - Not finite
  - No common diff^n exist
  - $n \geq 1$, P: prime
- Regular ⟺ D.F.A exist
- Minimal State DFA is always Uniq^u.
- DFA helps to write program easy.

Minimization Of DFA:

1. Remove inaccessible states.
2. Dead State is diff^t from ϕ
3. Merge equivalent State into Single State. → (F, F) or (NF, NF)

R_1: {A B F}$_{NF}$ {C D E}$_F$

R_2: {A B} {F} {C D E}

→ check two State with same input must belong to same grp. otherwise break it and check until two same Row Not encounter / generated.
→ Same Row of table can merge into Single.

Complement of DFA:

→ even a's ⇒ Not Even a's (odd)
→ Interchange
- NF → F } only.
- F → NF

NF = Non accepting.

$\bar{L} = \{0,1\}^* - L$

Note: (min' DFA is)

- exactly n length = n+2 state, (D),
- atleast n " = n+1 state, —.
- atmost n " = n+2 state, (D).
- divisible n = n state, —.
- product Rule: ** (or, also)

divisible n (?) exactly m (D)

n ⇕ (m+1) = n(m+1)+1

cross product of States ⇕ (m+2) - D

- No Dead state, No need to Substract
- Dead state, ignore then add only 1 at the End. *
- for 'n' states NFA we can have $\{1 - 2^n\}$ States of DFA *
- Subset Constr^n Algo **

NFA: $(Q, \Sigma, q_0, F, \delta)$

$\delta: Q \times \Sigma \to 2^Q$ (transition may be 0/1/2...)

→ easy Constr^n, No Min^z
→ No Complem
→ multiple state possb^e exist.

DFA ⇒ NFA, NFA ⇏ DFA

→ for valid string also automata may halt in Non-final state.
→ nth input is 'a' while reading from Right hand side = (n+1) state, NFA.

2^8 DFA state, 2^n state ⇒ DFA.

NFA to DFA:

- from NFA table, starting with first Row
- then only add newly state to Subsequent Row.
- Empty → Dead state.
- Single → same.
- multiple → new state. (combined)

→ doesn't effect langn of automata.
→ final DFA need not to be Minimal.

Expressive Power:

- No. of langn accepted by partr automata is known as Expressive power.

TM > LBA > PDA > F.A

NFA = DFA Expressive

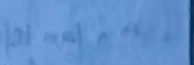
But NPDA > DPDA (Conv Not possb^l)

DTM = NTM

ϵ-NFA: $Q \times \Sigma \cup \{\epsilon\} \to 2^Q$

$a^n b^m c^k, m, n, k \geq 0$

- Easier than NFA



- ϵ-NFA → NFA:
  → Same States
  → Same initial State
  → State can go to final using only ϵ is also a final State.
  → for transition check all possibilities of $(q_0, a) = \{\ , \ , \ , \ , \ \}$ # a.ϵ = a, # ϵ.ϵ.a.ϵ.ϵ = a
  All / All input.
  → once NFA formed, use NFA-DFA then use minimization.

* Regular Expression :

- for one Regular langu many No. of regulas expression possb.
- But, one Regulas Exp can generate only one regulas langu.

language.	Expression.
$L = \{\}$	ϕ
$L = \{\epsilon\}$	ϵ
$L = \{a, b\}$	$(a+b)$
$L = \{aa, ab, ba, bb\}$	$aa+ab+ba+bb$
$L = \{\epsilon, a, a^2, a^3 \dots\}$	a^*
$L = \{a, a^2, a^3 \dots\}$	a^+
$L = \{a^n b^m \mid n \geq 1, m \geq 0\}$	$a^+ b^*$
$L = \{a^n 2^n \mid n \geq 0\}$	not possb
$L = \{a^n b^m \mid (n+m) \text{ is even}\}$	$(aa)^*(bb)^* + a(aa)^* b(bb)^*$

- $\epsilon + \epsilon = \epsilon$
- $\epsilon \cdot \epsilon = \epsilon$

similarly odd

- atleast 4 len String : $(a+b)^4(a+b)^*$
- exactly 4 " " : $(a+b)^4$.
- atmost 4 : $(a+b+\epsilon)^4$
- divisible by 4 : $((a+b)^4)^*$
- exactly 4 a's : $b^*ab^*ab^*ab^*ab^*$
- atmost 3 a's : $b^*(a+\epsilon)b^*(a+\epsilon)b^*(a+\epsilon)b^*$
- odd length palindrome : $a(aa)^*$
  - more than one Sym palindrome is Not Regl → only single character possible.
- a's divisible by 3 : $(b^*ab^*ab^*a)^* + b^*$
- Start & End with diff Symbols : $a(a+b)^*b + b(a+b)^*a$
- Substring aabb bba : $(a+b)^*(aabb + bbaa)(a+b)^*$
- 4th a from left : $(a+b).(a+b).(a+b).a(a+b)^*$
  - similar for Right.

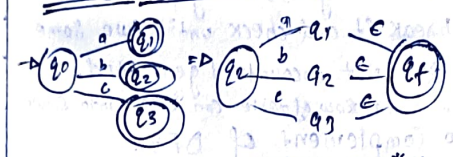
Note:

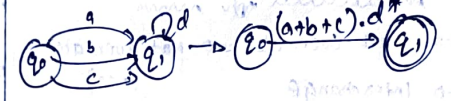
- all NFA states are accepting state then is $q_0 \in^*$ ⊗ !! accept
- odd len string of a's & b's = $(a+b)(a+b)^*$
- dead state is a NF final.
- $n \leq |w| \leq 2n-1$ for infinite L by F.A
- min DFA for Empty langu is 1 only (1 states)

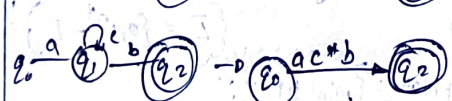
o properties

- $R + \phi = \phi + R = R$
- $R \cdot \phi = \phi \cdot R = \phi$
- $R + \epsilon = \epsilon + R \neq R$
- $R \cdot \epsilon = \epsilon \cdot R = R$
- $(R^+)^* = (R^*)^* = (R^+)^+ = R^*$
- $RR^* = R^+ = R^*R$
- $\epsilon^* = \epsilon$ } $\epsilon^+ = \epsilon$
- $\phi^* = \epsilon$ } $\phi^+ = \phi$
- $(a+b)^* \neq a^* + b^* \neq a^*b^* \neq (ab)^*$
- $(a+b)^* = (a+b^*)^* = (a^*+b)^*$
  - $= (a^*b^*)^* = (a^*+b^*)^*$
  - $= (b^*a^*)^*$
- $a^* + a^* = a^*a^* = a^*$
- $a+b = b+a$ } $ab \neq ba$
- $a + a^* = a^*$ } $a^+ + \epsilon = a^*$

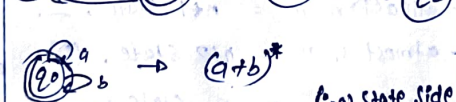
o Finite Automata → Regular Expr :

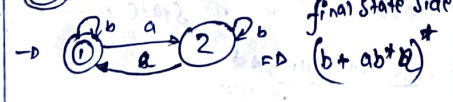


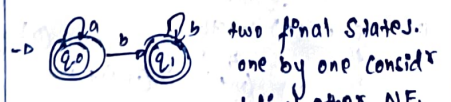


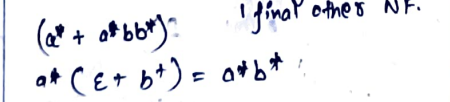












$(a^* + a^*bb^*)^*$

$a^*(\epsilon + b^+) = a^*b^*$

- ignore dead state while writing Regex. $(0^* + 0^+0^+) = 0^*$

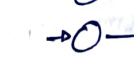
★ $(\epsilon)^* = \epsilon$ } $\phi \cup \epsilon = \epsilon$

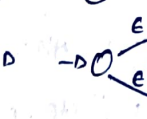
★ $(\phi)^* = \epsilon$

★ $\{\}$ ⊗ = empty language is not Reg ⊗ wrong concept.

ϕ ⇒ No edge to final state

ϵ ⇒ 

a ⇒ 

$r_1 + r_2$ ⇒ 

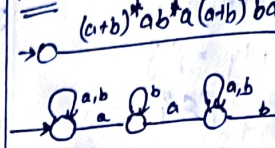
$r_1 \cdot r_2$ ⇒ 

r^* ⇒ 

Regex ⟶ ∈ Nfa.

$L \neq L^c$: Same min DFA.

Note:



- min DFA : $(a+b)^*aba(a+b)^*$ (n+1) state. → n len substring

- → $(a+b)^*aba$. n len ending ⇒ (n+1) States
- → $aba(a+b)^*$. n len starting ⇒ (n+2) States
- → $(a+b)(a+b)\ldots(n)$. exactly 2 len ⇒ (n+2) States
- → atleast n ⟹ $(n+1)$.
- → divisible n ⟹ n
- → atmost n ⟶ $n+2$
- → n^{th} Symbol is 'a' ⟶ 2^n states from Right/End
- → from left ⟶ $(n+2)$
- → $a^*b^*c^* \ldots n^{th^*}$ ⟶ $(n+1)$
- → $(a+b)^+$ ⟶ 2

★ ---

- → $n_a \leq 1$ → 3 → 2 ∴ 6+1(dead)
  $n_b \leq 2$ → 4 → 3 ↓ all final
- → $(aaa + aaaa)^*$ → 7 state.
  - divisible by 3 or 4
  - $(a+b)^*b(a+b)$ 2 len end^n : n+1 : 3rd

★ $\{a^{mn} \mid n \geq 1,\ m > n\}$ Regular

$\{a^2, a^3, a^4 \ldots\} = (aa)a^*$

o Min Pumping length :

→ $a^*b^*b(cd)^*$ → min len String $n=1$

Min pump^n len ⇒ n+1 ⇒ 2

→ $p_1 = 2$ ∪ $p_2 = 3 = 3$ (max)

- Pumping Len = $|w|_{min} + 1$

* Regular Language :

- finite ⇒ always Reg
- infinite = depency → ✗
- { } = empty / epsilon always Reg.

- $a^n b^{2m} c^{3k}$; n,m,k ≥ 0 ✓ equal diff
- $a^n b^m c^{k^3}$; " ✗ expo
- a^{2^n} $\{a^{n^n}\}$ a^p $\{a^{n^2}\}$ → Not Regular
- $(a^p)^*$ / p = prime No. → Regular. Since, we can make a^*.
- a^k → Regular. $\{a^{even}$ → Regular.
- $(a^{n^n})^*$ $\{(a^{2^n})^*$ $\{(a^n)^*$ → Regular.
- set of balanced paranthesis, equal No. of ( ), Even len palindrome of English letter, odd also ⇒ all Not Regular.

- $L = \{ww^R \mid w \in (a+b)^*\}$ ⇒ Non Regular
- $L = \{ww^R \mid w \in a^*\}$ ⇒ Regular
- $L = \{wxw^R \mid w \in (a+b)^*, x \in a+b\}$ ⇒ Non Regular
- $L = \{wxw^R \mid w,x \in (a+b)^*\}$ ⇒ Regular.
- $L = \{a^n w a^n \mid w \in (a+b)^*\}$ ⇒ "
- $L = \{ww^Rx \mid w,x \in (a+b)^*\}$ ⇒ Regular.
- $L = \{xww^R \mid w,x \in (a+b)^*\}$ ⇒ Regular.
- $L = \{wxw^R \mid w,x \in (a+b)^+\}$ ⇒ Regular. $a(a+b)^*a + b(a+b)^*b$
- $L = \{ww^Rx \mid w,x \in (a+b)^+\}$ ⇒ Non Regular.
- $L = \{wwx \mid$ " $\}$ Non Regular.
- $L = \{wxw \mid$ " $\}$
- $L = \{xww \mid$ " $\}$
- $L = \{xww \mid w,x \in (a+b)^*\}$ Regular, $(a+b)^*$ exist
- $L = \{wxw \mid$ " $\}$ ✗ can be in anything
- $L = \{wwx \mid$ " $\}$
- $L = \{x \mid n_a(x) = n_b(x)\}$ ⇒ Non Regular.
- $L = \{x \mid n_a(x) > n_b(x)\}$ ⇒ " "
- $L = \{x\$y \mid x,y \in (a+b)^*\}$ ⇒ Regular.
- No Dependency ⇒ Regular.
- $L = \{wxw, w \in (a+b)^+, x \in (a)^+\}$ Not Regular.
- $L = \{wxw, w \in (a)^+, x \in (a+b)^+\}$ Regular $a(a+b)^+a$

Note:
$S = 11^*0$, $t = 1^*0$ ⇒ $L(S) \subset L(t)$
$\delta = (1+0)^*$ ⇒ $L(S) \in L(\delta)$

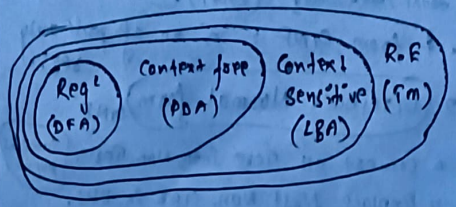
○ Closure properties : (for Regular)

1. Subset : may or may not be Regu
2. Union : Always Regular.
   $(L_1 \& L_2) \Longrightarrow (L_1 \cup L_2)$ is Regular.
   - Infinite Union of $L_1 L_2 L_3 \ldots$ Regular Not Regular. (may / may not)
3. Infinite Intersection : Not Regular.
4. Complement. (Not closed) $L_1 \cap L_2 \Rightarrow L_1 \cap L_2$ (R ⇏ R)
   L (Reg) ⟺ L^c (Reg)
   $(L^c)^c = L$ $L^c = \Sigma^* - L$
5. Concatenation ✓ (Reg)
   ⇒ $L_1 . L_2$ is Reg. (Reg) (Reg)
6. $L_1 \cap L_2$ ⇒ Reg (Reg) (Reg)
   - $\epsilon \cap A$ = always
   - $\phi \cap A$ = Reg.

   $L_1 = (a+b)^*a$, $L_2 = (a+b)^*b$ ⇒ $L_1 \cap L_2 = \{\}$ → Reg. empty → 1 min DFA state.
7. Reversal :
   if L Reg. then L^R is also Reg.
   - $(L^R)^R = L$ ⟺
8. Kleene Closure :
   if L_1 is Reg then L^* is Reg also.
   Same for L^+ ⇏
   * Not closed means may or may not.

* Chomsky Classification :

Language	Automata	Grammar
Regular L	F.A	Regular Gr
C.F.L	PDA	CFG
C.S.L	LBA	CSG
R.E.L	TM	REG



→ for every lang^y ⇒ grammar exists
& every grammar ⇒ gen^s one lang^y
→ finite ∩ Non-Regular = finite ⇒ Reg^L

→ $(a+b)^*(bb)^* = (ab)^*(a+b)^*$
→ $(aL+ba)^* = (ab(ab)^* + ba(ba)^*)^*$

* Grammar :

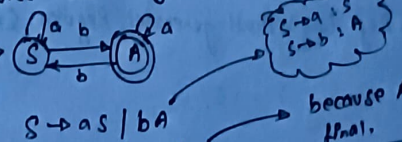
→ S, A, B, C ... → Non termin
→ a, G, b, c, d → termin
→ set of product^n → deriv^n

- $S \to aS/bS/\epsilon$: $(a+b)^*$
- $S \to as/a$: a^+
- $S \to aS/\epsilon$: a^*
- $S \to aS/bS/a/b$: $(a+b)^+$
- $S \to aSb/\epsilon$: $(a^n b^n \mid n \geq 0)$
- $S \to AB$, $A \to aA/\epsilon$, $B \to bB/\epsilon$ } (a^*b^*)
- $S \to AB$, $A \to aAb/ab$, $B \to cB/c$ } $\{a^n b^n c^m, n,m \geq 1\}$
- $S \to as/Sb/ab$ → $a^*(a+b)b^*$
- $S \to aAa/bAb/a/b$, $A \to aA/bA/\epsilon$ } Start^n & End^n with same Symbols.
- $S \to asa/bsb/a/b$ } odd length palindrome
- $S \to AB/aAbB$, $A \to aaA/\epsilon$, $B \to bbB/\epsilon$ } $a^n b^m$. n+m ∈ Even
  $(aa)^*(bb)^* + a(aa)^*b(bb)^*$
- $S \to AbB/aAB$, $A \to aaA/\epsilon$, $B \to bbB/\epsilon$ } $a^n b^m$. n+m ∈ odd
  $(aa)^*b(bb)^* + a(aa)^*(bb)^*$

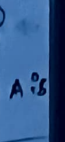
○ Types of Grammar :

- Regular Gr ⇒ (either Not both) $A \to xB/x$ ← RLG, $A \to Bx/x$ ← LLG
- Context F. Gr ⇒ $A \to \alpha$ → any → Single N.T.
- Context S. Gr ⇒ $\alpha \to R$ → $|\alpha| \leq |R|$ any.
- Unrestricted Gr ⇒ $\alpha \to R$ any.

○ F.A to Regular Grammar :

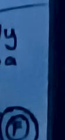


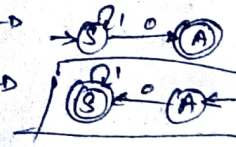
$S \to aS/bA$
$A \to bS/aA/\epsilon$

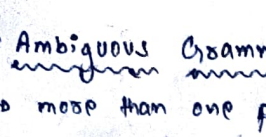


because A is final.

Same Grammar to F.A.
- S → aS means: S —a→ S
- S → bA means: S —b→ A



- S → S1 | 0 (01*)
  - S → 1S | 0 reverse.
    - 

      Now reverse this
    - 

      01*

Ambiguous Grammar:

- more than one parse tree,
- more than 1 LMD / 1 RMD.

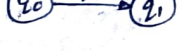
E → E+E | E*E | n → Ambiguous.

S → AA
A → aA | a } Amb

- Ambiguity prob^{l} is undecidable prob^{lm}
  - No algo exist to check ambiguity in grammar.
- Removing Ambiguity is undecidable problem also.
- S → SS | ∈ (ambiguous)
- S → aSbS | bSaS | ∈ (〃)
- S → aSb | ab → (Not Ambiguous)
- S → AaAb | BbBa
  A → ∈
  B → ∈ } → (Not Ambiguous)
- S → AB | BC
  A → BA | a
  B → CC | b
  C → AB | a } (Ambiguous)

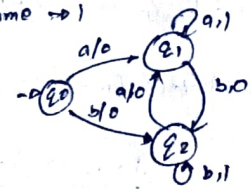
* Mealy & Moore:

- F.A. with output → Deterministic, No final State
- Mealy — output with transact^{n}
- Moore — output with State.





* Last two input Same → 1
  otherwise → 0



⇒ Reg^{l} langu Must generates Regular Exp^{n}.

* Simplifications of CFG:

1. Removing NULL Prod:

S → AaB
A → aA | ∈ *
B → b | ∈ *

Null → Unit prod → Useless N.T.

→ on removing A & B will effect, put ∈ respectively one-by-one on each prod where A & B are on RHS.

S → AaB | aB | Aa | a
A → aA | a
B → b

(all poss^{b} comb^{n}; direct removed.)

2. Removing Unit production: (only one N.T alone.)

- S → Aa | B
- B → A | bb
- A → a | ba | B

always start with bottom to top.

A → a | ba | A | bb (✗ discard)
B → a | ba | bb | bb
S → Aa | a | ba | bb

3. Removing Useless Non Terminal:

→ Any N.T. is not gener. String.
→ Any N.T. is not req^{r} for deriv.
→ If any N.T removed (unreachable, loop) then the entire product^{n} in which that N.T present will remove. (means it will acts as ∅)*

Normal form of CFG: (CNF)

1. Chomsky N.F → only two N.T.
   - A → BC
   - A → a → single.

→ S → asb/ab
  S → ASB/AB } ⇒ S → DB/AB
  A → a (N.T) } P → AS
  B → b } A → a
  D → b (Break^{n})

- To gen^{r} 'n' len string from CNF grammar, total deriv^{n} req^{d} (2n−1).
- # from CNF: 'n' no. of prdc only req^{d}.

2. Greibach Normal form GNF
   → convert all term into Non. Term. (same as poems)
   → Replace first Non. Ter in RHS.

S → ASB/AB } ⇒ S → aSB/aB
A → a
B → b
A → a ✗ (if Term exist can be made.)

* Note:
→ Decidable problm in CFG: 1. Emptyness, 2. finiteness, 3. membership.
(If Recurs^{n} then Not. passe time)

* Push down Automata:

- PDA > F.A (used in pr...)
- PDA = F.A + Stack
- PDA → Regular langu.
  accept → Context free langu. (CFL)
- Size of stack is ∞.
- PDA: (Q, Σ, $, q_0, F, Z_0, Γ) → Clos... Stack
  ↑ initial Stack top
- Q × Σ ∪ {∈} × Γ → Q × Γ*

⇒ String Accepted by either Stack Empty OR q$_f$ final State. Enter.

⇒ Both Method accepts Same No. of language. & (both must follow (for an accepted langu)

NPDA > DPDA → default.

Expressive powers

⇒ PDA with finite stack Accepts only Regular langu. (No PDA, NO CFL)

- Construction of PDA:

→ If we move to next State, it means we cannot accept ∈ of prev state.

→ NPDA ≠ DPDA
- Every NPDA cannot be converted into DPDA

- push: δ(q0, a, Z0) → (q0, aZ0)
  (state, input, top → nxt state, top (L-R))
- pop: δ(q0, b, a) → (q1, ∈)
  (input b top → nxt state, b pop)

→ L = {a^{n}b^n | n, m ≥ 1}
→ push pop. Empty accepted ✓

→ L = {a^{n}b^{n}c^m | n, m ≥ 1}
→ push, pop → skip + reading. Empty accepted

→ L = {a^{n+m} b^{n}c^m | n, m ≥ 1}
push pop. empty. accepted ✓

→ L = {a^n b^{n+m} c^m | n} acc ✓
push pop (a) push (c) pop. Empty

→ L = {a^n b^m | n ≥ m, m ≥ 1} acc. ✓ (on empty/a)
→ L = {a^n b^m | n ≥ m, n, m ≥ 1}

- $L = \{ n_a = n_b \mid x \in (a+b)^* \}$
  - push for same alpha, initial
  - pop for diff alpha, $\epsilon \to$ accept.
  - (only single state) Req'
- $L = \{ n_a > n_b \mid x \in (a+b)^* \}$
  - two state Req' :
    - $\to$ $a, a/\epsilon$ goes to q_2
- $L = \{ n_a \geq n_b \mid x \in (a+b)^* \}$
  - single state : $\epsilon, a/\epsilon \to$ pop also.
    $\epsilon, z_0/\epsilon \to$ final

* we can push/pop other symbols also. like $(z_0, a/x)$

* $a^n b^{kn} \mid n \geq 1$ → CFL, PDA exists.
→ arithmatic common diff^n.

o Where PDA Not possible :

- $a^n b^{2^n}$: No common diff^n
- $a^{n!} b^{n!}$: Not possible.
- $a^n b^n c^n$: multiple comparisons.
- $a^n b^n c^{2n}$: " "
- $a^n b^m c^n d^m$: stack is LIFO!!
  - $a^n b^m c^m d^n$: a^n b^m c d ✓
  - wcw : LIFO. $\{ ww \}$ (Not possible)

o NPDA v/s DPDA :

- $wcw^R \mid w \in (a+b)^*$: → push until c then $(a,a), (b,b)$ pop
  (odd len palindrome) DCFL & CFL Both.
- $ww^R \mid w \in (a+b)^*$: .CFL but not DCFL
  .NPDA ✓ DPDA ✗
  (3 state) DPDA
- $L = \{a^n b^n\} \cup \{a^n b^{2n}\}$: NPDA Combined.
  .CFL but not DCFL
- $\{a^n b^n c^m\} \cup \{a^m b^n c^n\}$: NPDA / CFL but not DCFL

→ Every DPDA is NPDA but Every NPDA is not DPDA.

Same, ∀ DCFL ⇒ CFL
∀ CFL ⇏ DCFL

→ (odd len palind)^c → Even len palindrome
↳ CFL, DCFL ✗ ✓

→ $\{a^m b^n c^k \mid$ if m=even $(n=k)\}$
→ $\{a^n c b^n\} \cup \{a^n d b^n\}$ DCFL

o CFG → PDA :

1. $\delta(q_0, \epsilon, z_0) \to (q_1, Sz_0)$
2. for Every N.T. —— (N.T. → α)
   - $\delta(q_1, \epsilon, \text{N.T}) \to (q_1, \alpha)$ push L→R
   - $\delta(q_1, T, T) \to (q_1, \epsilon)$ pop for same.
3. $(q_1, \epsilon, z_0) \to (q_f, z_0)$ accept.

- $S \to a\alpha \mid b$: NPDA. → always
  (push or push) (may/may not DPDA)

o CFL v/s Regular :

→ If Regular then CFL also.
→ If CFL then mayor maynot Reg^L.
- If finite then Reg^L ⇒ CFL.

1. $a^n b^n c^m \mid m \neq n$ ⇒ CFL ✗.
2. $a^n b^{n+4}$ ⇒ CFL ✓ but not Reg^L
3. $a^n b^{Kn+c}$ ⇒ CFL ✓ but not Reg^L
4. $a^m b^n \mid m \neq n$ ⇒ CFL ✓ "
5. $a^n b^{n^2}$ ⇒ No CFL ✗
6. $a^{n!} b^{n!}$ ⇒ No CFL ✗
7. $a^n b^m c^{n+m}$ (push / pop) ⇒ CFL, Not Reg^L
8. $a^n b^{n+m}$ ⇒ CFL, " "
9. $a^{m^2} b^{n^3} c^{k^5}$ ⇒ CFL ✗
10. $a^{3^n} b^{5^k} c^{2^l}$ ⇒ CFL ✗
11. $a^n b^{n+m} c^m$ ⇒ CFL ; but no Reg^L
12. $a^n b^m c^L \mid n>m$ or $m>L$ ⇒ CFL
13. $a^i b^j c^k \mid i>j>k$ ⇒ CFL ✗
14. $a^i b^j c^k \mid j = i^2 + k^2$ ⇒ CFL ✗
15. $a^i b^j c^k d^L \mid i=l$ or $j=k$ ⇒ CFL ✓
16. $a^i b^j c^k d^L \mid$ " & " ⇒ CFL ✗
17. $a^{2^n} / a^{n^2} / a^p$ ⇒ CFL ✗
18. a^{2n+1}, a^{kn} ⇒ CFL ✓
19. $wxw \mid w \in (a+b)^*$ ⇒ CFL ✗
20. $wxw \mid w, x \in (a+b)^*$ ⇒ Reg ⇒ CFL
21. $ww^Rx \mid w, x \in (a+b)^+$ ⇒ CFL, but not Reg^L
22. $ww \mid w \in (a+b)^+$ ⇒ CFL ✗
23. $ww^Rw \mid$ " ⇒ "
24. $ww^Rww^R \mid$ " ⇒ "
25. $x \mid x \in (a+b)^m$ ⇒ Reg^L ⇒ CFL
    ($n_a \bmod 5 = 0$ & $n_b \bmod 4 = 0$)
26. $a^n b^{2n} c^{3n}$ ⇒ CFL ✗
27. $a^n b^n c\, a^m b^m$ ⇒ CFL, but not Reg^L
28. $a^n b^m c^k \mid n \neq m$ or $m \neq k$ ⇒ CFL, but not Reg^L
29. $a^{2n} b^n c\, a^{2m} b^m \mid n \geq 1$ ⇒ CFL ✗
30. all balanced parenthesis ⇒ CFL
31. odd len palind ⇒ CFL
    even len " "
    of English alphabet.
32. $a^m b^m c^{m+n}$ ⇒ CFL ✗

* Closure properties of CFL :

1. Subset : Not Closed. (C) DCFL ✗
2. Union : closed DCFL ✗
   $L_1 \to$ CFL, $L_2 \to$ CFL ⇒ $L_1 \cup L_2$ = CFL
   $L_1 \to$ DCFL, $L_2 \to$ DCFL
3. Intersection : Not closed ⇒ DCFL ✗ But CFL ✓
   Same for DCFL: $L_1 \cap L_2$ ⇒ DCFL ✗ But CFL ✓
4. Intersection with Regular : ⇒ DCFL closed
   But DCFL ∧ Reg = DCFL closed
5. Complement : Not closed
   But $DCFL^c \to DCFL$ = closed
6. Difference : Not closed. DCFL → ✗
   $L_1 - L_2 \Rightarrow L_1 \wedge L^c$
7. Difference with Regular : Closed
   Same for DCFL: $L_1 - Reg = L_1 \wedge Reg^c$ = CFL always
8. Concatenation : Closed, But DCFL → ✗
9. Union with Regular : closed, DCFL → ✓

	CFL	DCFL
10. Kleene :	✓	✗
11. positive :	✓	✗
12. Reversal :	✓	✗

* x ∧ Regular = x always

* Not closed means may or may not.

⇒ CFL (Not closed) in : Intersection, Compl^n, diff^n.

⇒ DCFL (closed only) in : Complem, L-Reg, L ∧ Reg, Inv HM.

* Turing Machine :

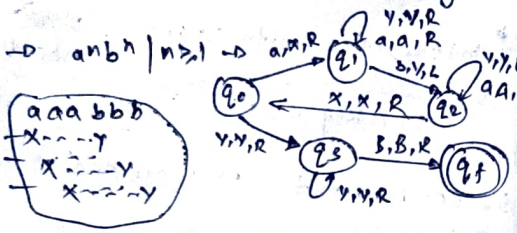
- infinite one ended tap, can move L or R, can Read or write.

$a^n b^n c^n$ / wcw

- problem not solved by turing machine are not solved by computer itself.
- $(Q, \Sigma, q_0, F, \delta, B, \Gamma)$ → tape alpha; Blank
- $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ left/right.
- we have to study turing machine as a language Recognizer.

→ Halt at final State : accept
Non final " : Reject
Loop : can't Say.

→ $a^n b^n \mid n \geq 1$ →



○ Recursive language (Decidable) :

- TM will always halt. → F : accepted
  → NF : Rejected
→ Turing decidable langᵘ. as well as Recognize

○ Recursive Enumerable langᵘ : (UD)

- if string valid ⇒ halt at final state
- if not ⇒ halt at NF state (OR) infinite loop ∞
- Turning Recognizable Machine. langᵘ. (undecidable langᵘ)

#Note:

- Every Recursive language is REL but Every REL need not be recursive.



⇒ R→L→R→L ... Loop.
∴ $L = \{\} \neq \Sigma^*$. → Not even for valid string.

- Non REL : (No T.M), uncountable set.
- Any modification of turing machine doesn't effect the expressive power of turing machine.
- Decidable prob' : ∄ these exist halting T.M.
- UD : if ∄ no halting T.M.
- To proove problem UD we use Red^c Technique.

○ Reduction :

if A is reducible to B then;

$A \leq B$ if B is yes : then A is also yes :

- if A is No : then B is also No : (Y ← Yes, N → N)

○ Closure propᵖ of Rec & REL:

	REC	REL
• Union	✓	✓
• Concat	✓	✓
• Intersect	✓	✓
• Compliment	✓	✗
• Difference	✓	✗
• L ∧ Reg	✓	✓
• L − Reg	✓	✓
• Kleene/+ve	✓	✓
• Substitution	✗	✓
• Homeomorphism	✗	✓
• Reverse	✓	✓
• Inverse HOMEO	✓	✓

→ Rel^c = may or may not REL
if (REL & not Rec)^c ⇒ Non REL (always)

#Note:

- F.A + 1 stack = PDA
- F.A + 2 stack = T.M } Same expressive powers
- F.A + 10 stack = T.M. }
- only computing speed may increase.

→ Complement of finite langᵘ ⇒ infinite.

- L → Regular, M → Not " } L, M ⇒ may or may not (not necessarily Non Regular)

→ infinite Regᴸ contains undecidable as subset.

→ L → CFL ; Σ* → not CFL but CSL, Rec, REL ✓

○ undecidability :

→ REL is Decidable only in intersection of two langᵉ is same type

→ Rec is Decidable only in.
- w ∈ L (membership). —
- Intersection same type? —
- ~~union~~ compliment same type? —

→ CFL (Non Deterministics) only
- w ∈ L ? (membership)
- L = ϕ ? (emptiness)
- L = finite ? (finiteness)

→ apart from these 3, DCFL can also detects :
- $L_1 = L_2$? two automata accept same L?
- $L = \Sigma^*$? completeness
- L^c finite ? compliment finiteness
- L^c same type?
- L is Regular ?

→ Regular is always Decidable.